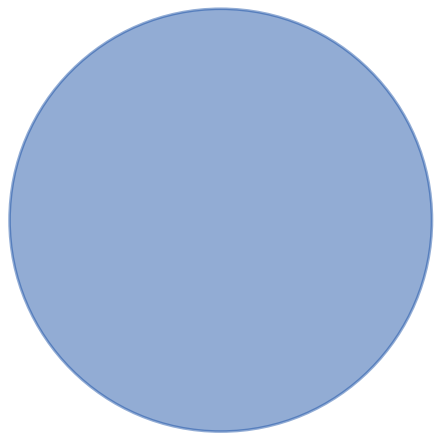


Adaptare in jocuri video



AI care modifica comportamentul in functie de stilul jucatorului

De ce este importanta adaptarea?

Abilitati diferite

Jucatori de toate nivelurile – de la incepatori la experti

Personalizare

Fiecare jucator are un stil unic de joc si preferinte

Evitarea plictiselii

Un joc prea usor devine neinteresant rapid

Retentie crescuta

Adaptarea creste durata de sesiune si fidelizarea

Categorii de adaptare

Dificultate

- Ajustarea vitezei inamicilor
- Numarul de obstacole
- Resursele disponibile (viata, munitie)
- Agresivitatea AI-ului

Comportament inamici

- Strategii de atac adaptate
- Detectia si urmarirea jucatorului
- Timp de reactie dinamic
- Coordonarea grupurilor de inamici

Mediu

- Distributia resurselor pe harta
- Layout-ul nivelului
- Generare procedurala adaptiva
- Evenimente de joc dinamice

Naratiune

- Ramificarea povestii
- Dialoguri generate contextual
- NPC-uri cu memorie
- Finaluri multiple personalizate

Algoritmi utilizati in IA adaptiv

Algoritmi Genetici

- Evolutia parametrilor de comportament
- Optimizare prin selectie naturala
- Fitness calculat din metrice jucator

Dynamic Difficulty Adjustment

- Ajustare in timp real a dificultatii
- Bazat pe performanta jucatorului
- Explicit, implicit sau hibrid

Reinforcement Learning

- Agent care maximizeaza recompensa
- Q-Learning clasic si Deep Q-Learning
- Aplicat in DDA si comportament NPC

Case-Based Reasoning

- Memorarea si reutilizarea cazurilor
- Ciclul Retrieve-Reuse-Revise-Retain
- Adaptare la stilul de joc individual

Algoritmi Genetici

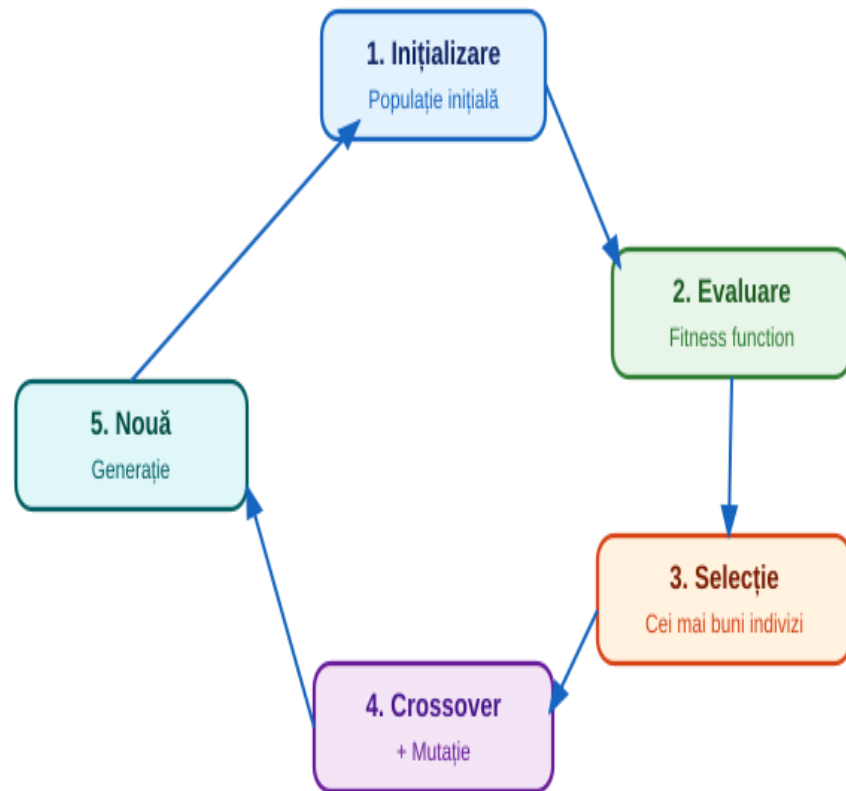
Evolutie pentru optimizarea parametrilor de joc

Algoritmi Genetici

Algoritmi genetici metaheuristici de optimizare inspirati din evolutia biologica (selectie naturala, incrucisare, mutatie). Sunt folositi pentru a evolua automat parametrii comportamentului AI in jocuri.

In jocuri - optimizeaza:

- viteza si agresivitatea inamicilor
- dificultatea nivelului
- strategiile de atac
- durata si resursele disponibile



Algoritmi Genetici - Parametri evolutivi

Parametri de dificultate

- Viteza inamicilor
- Numar de obstacole
- Resursele disponibile
- Durata nivelului
- Timpul limita

Parametri comportament AI

- Agresivitate
- Detectare jucator
- Timp de reactie
- Raza de vizibilitate
- Coordonare grup

Parametri narativi

- Frecventa evenimentelor
- Ramificari poveste
- Tonul dialogurilor
- Densitatea obiectivelor

Algoritmi Genetici - Evaluarea Performantei

Functia Fitness

$$\text{fitness} = \alpha \times \text{scor_jucator} - \beta \times \text{numar_esecuri} + \gamma \times \text{timp_supravietuire}$$

α = ponderea scorului | β = penalizarea esecurilor | γ = bonus supravietuire

Metrici de performanta utilizate:

- Rata de succes in misiuni
- Frecventa erorilor (morti, restart)
- Timpul de finalizare a nivelului
- Feedback explicit (rating jucator)

Algoritmi Genetici - Evolutia comportamentului inamicilor

Procesul de evolutie

- Generatia initiala: parametri aleatori pentru fiecare inamic
- Evaluare: cât de provocator a fost inamicul?
- Selectie: pastram cei mai buni (elitism)
- Crossover: combinam parametrii parintilor
- Mutatie: variatie aleatoare mica (~5%)
- Noua generatie → iteram

Rezultatul evolutiei

- Jucatori mai buni → inamici mai rapizi, mai inteligenti
- Jucatori in dificultate → inamici mai lenti, mai previzibili
- Personalitate AI emergenta, unica per jucator
- Exemplu: Alien: Isolation — AI-ul Xenomorphului se adapteaza la comportamentul jucatorului (tehnici de ascundere, rute preferate)

Algoritmi Genetici - Avantaje si Provocari

Avantaje

- Personalizare automata – fara reguli scrise manual
- Scalabil la multi parametri simultan
- Capacitate de invatare pe termen lung
- Adaptare individualizata per jucator
- Nu necesita date de antrenare etichitate

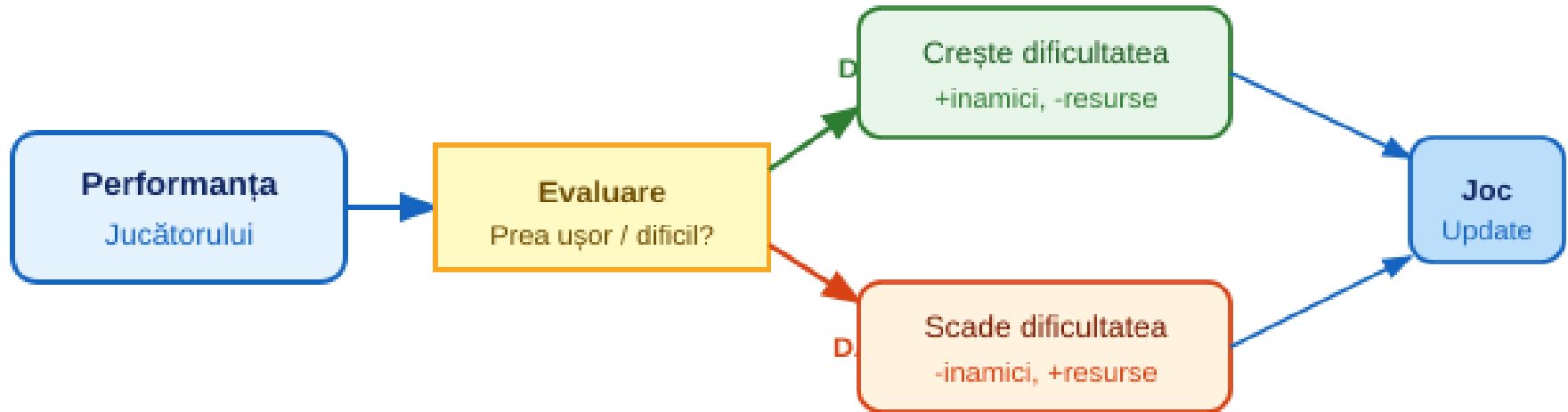
Provocari

- Definirea unei functii fitness realiste si corecte
- Timp de convergenta – poate fi lent in timp real
- Exploatare vs. explorare (risc de overfitting)
- Costuri de calcul in aplicatii real-time
- Posibila manipulare a functiei fitness de jucatori

Dynamic Difficulty Adjustment

Dynamic Difficulty Adjustment (DDA)

DDA este o tehnica prin care dificultatea unui joc este ajustata automat, in timp real, in functie de performanta jucatorului, cu scopul de a mentine o experienta echilibrata



Tipuri de DDA

Explicita

- Jucatorul alege nivelul de dificultate (Easy/Normal/Hard)
- Control total al jucatorului
- Previzibila si transparenta
- Dezavantaj: necesita decizie constienta

Implicita

- Jocul ajusteaza fara a informa jucatorul
- Experienta fluida si naturala
- Jucatorul nu realizeaza ca sistemul intervine

Hibrida

- Combina abordarile explicite si implicite
- Jucatorul seteaza un nivel de baza
- Adaptare in timp real
- Flexibilitate maxima

DDA

Componente cheie

- Evaluarea performantei jucatorului
- Detectia dificultatii percepute
- Mecanismul de ajustare a jocului
- Feedback si reechilibrare continua

Parametri de evaluare

- Timp de reactie al jucatorului
- Numar de vietii utilizate
- Precizia loviturilor
- Rata de succes in misiuni/obiective
- Durata sesiunii de joc

DDA - Exemple de ajustare

Jucator in dificultate

- AI mai slab si mai lent
- Mai putini inamici pe ecran
- Resurse mai dese (munitie, viata)
- Indicii suplimentare in mediu

Jucator echilibrat

- Parametri stabili, fara modificari
- Mentinerea tensiunii optime
- Eveniment special rar pentru varietate
- Recompense proportionale cu efortul

Jucator expert

- AI mai inteligent si agresiv
- Mai multi inamici simultan
- Resurse rare, provocare maxima
- Obiective suplimentare si secrete

DDA

Actualizarea dificultatii

$$D(t+1) = D(t) + \alpha \cdot (P_{\text{target}} - P_{\text{current}})$$

$D(t)$ = dificultatea curenta

P_{target} = performanta tinta (de ex. 60% rata de succes)

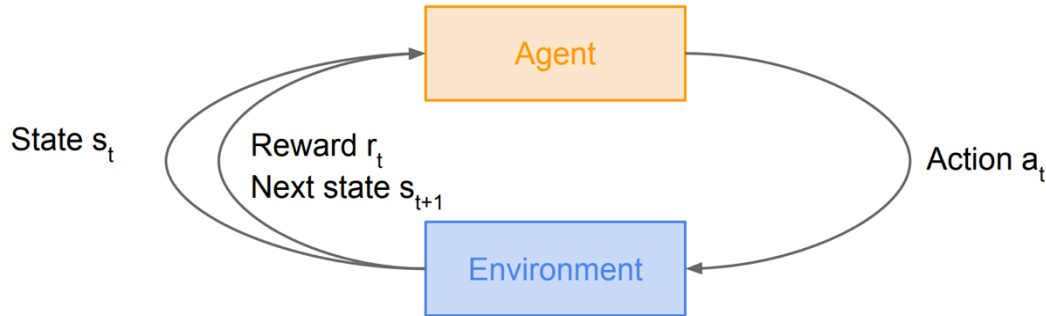
P_{current} = performanta masurata a jucatorului

α = rata de ajustare (learning rate, ~0.1–0.3)

→ Se aplica iterativ dupa fiecare nivel sau sesiune de joc

Invatare prin recompensa

Reinforcement Learning



Agent

AI-ul care controleaza dificultatea sau comportamentul

Mediu

Jocul si jucatorul – sursa de feedback

Actiune

Ajustarea parametrilor: viteza, nr. inamici, resurse

Recompensa

Metrici de satisfactie / performanta ale jucatorului

Reinforcement Learning aplicat in DDA

Stare (S)

- Nivelul curent de dificultate
- Scorul si vietile ramase ale jucatorului
- Timpul de reactie (ultimele 10 actiuni)
- Rata de succes pe ultimele n niveluri
- Numarul de restart-uri consecutive

Actiuni (A)

- Scade / creste dificultatea (step mic)
- Schimba comportamentul adversarului
- Modifica distributia resurselor

Recompensa (R)

- +1 daca jucatorul ramane angajat / avanseaza
- -1 daca jucatorul abandoneaza / pierde interesul
- 0 stare neutra (mentinere echilibru)

Reinforcement Learning

Avantaje

- Personalizare in timp real fara interventie umana
- Scalabilitate la multiple stiluri de jucatori
- Adaptabilitate la jocuri open-world si multiplayer
- Capacitate de a descoperi strategii neasteptate
- Imbunatatire continua pe masura acumularii datelor

Provocari

- Definirea functiei de recompensa potrivite
- Evitarea manipularii de jucatori
- Costuri mari de antrenare pentru modele complexe
- Echilibrul intre provocare si frustrare
- Instabilitate in antrenament (varianta mare)

Limitările Q-Learning clasic

Problema spatiilor de stare mari

- RL clasic (tabel Q): functioneaza in spatii mici – ex. navigarea unui labirint simplu
- Jocuri reale: spatii enorme – ex. navigarea unui oras virtual cu mii de stari posibile → tabelul Q devine impracticabil (memoria si viteza esueaza)

Problema generalizarii

- Valorile Q invatate pentru o stare nu se transfera la stari similare
- Ex.: daca agentul a vazut inamicul la stânga, nu stie sa reactioneze când e la dreapta

Solutia: Deep Q-Learning

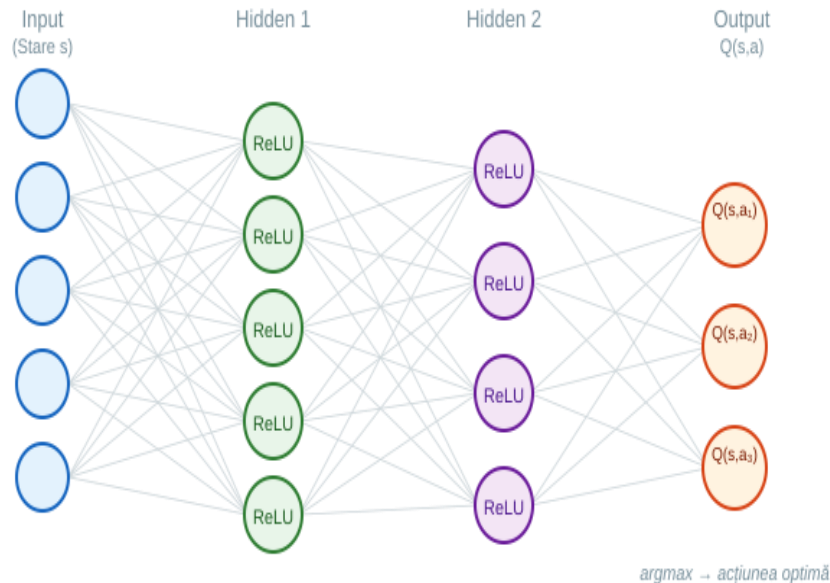
- Tabela Q se inlocuieste cu o retea neurala profunda (DNN) care aproximeaza $Q(s,a)$ → generalizare intre stari similare

Deep Q-Learning - Introducere

Deep Q-Learning (DQL) inlocuieste tabelul Q cu o **retea neurala** care aproximeaza functia $Q(s,a)$ pentru orice stare, inclusiv stari nevazute anterior.

Arhitectura DNN

- Input: vectorul de stare (caracteristici jucator + joc)
- Straturi ascunse: cu activare ReLU
- Output: valorile Q estimate pentru fiecare actiune posibila
- Actiunea aleasa: $\operatorname{argmax} Q(s,a)$
- DNN-ul poate generaliza intre stari similare



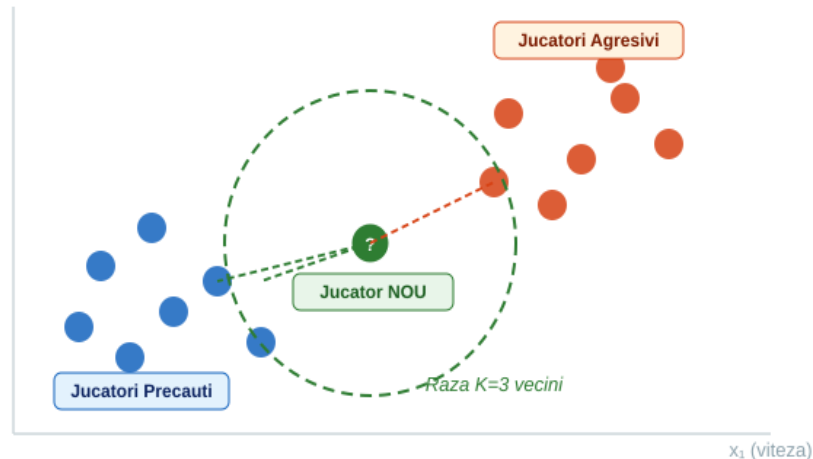
Case-Based Reasoning

Adaptare prin reutilizarea cunostintelor din situatii anterioare

K-Nearest Neighbors

KNN (K-Nearest Neighbors) invatare pentru decizii pe baza similitudinii cu cele mai apropiate K exemple din setul de date.

"Similar cu vecinii cei mai apropiati din spatiul caracteristicilor"



Clasificare

Ce tip de jucator esti?

Regresie

Ce dificultate i se potriveste?

Regasire

Ce continut i-ar placea?

Spatiul caracteristicilor

Date jucator reprezentate ca vector

- Fiecare jucator = un punct in spatiul n-dimensional
- Fiecare dimensiune = o caracteristica masurabila
- Similitudinea = distanta mica intre puncte

Exemple de caracteristici in jocuri

- Viteza medie de deplasare (normalizata 0-1)
- Rata de succese (headshots, combo-uri)
- Timp mediu pe nivel / sesiune de joc
- Frecventa utilizarii abilitatilor speciale
- Preferinta stil: agresiv / defensiv / explorator

Vectorul caracteristici al unui jucator:

```
x = [viteza, precizie, sesiune, abilitate, stil]
```

```
x_A = [0.82, 0.65, 45, 0.40, 0.72] (jucator agresiv, rapid)
```

```
x_B = [0.30, 0.90, 90, 0.70, 0.20] (jucator precaut, metodic)
```

Caror jucatori seamana cu jucatorul X?

KNN: Clasificare, regresie si regasire

Clasificare

- Predictie: clasa din care face parte jucatorul
- Vot majoritar intre K vecini
- Ex: Tip jucator (agresiv / precaut / explorator)
- Output: eticheta discreta

3 din 5 vecini = Agresiv =>
Clasa: Agresiv

Regresie

- Predictie: o valoare numerica
- Media (sau media ponderata) a vecinilor
- Ex: Dificultate optima (0.0 - 1.0)
- Output: valoare continua

Vecini: 0.6, 0.7, 0.65 =>
Medie: 0.65

Regasire

- Gaseste cele mai similare K exemple
- Ex: Cele mai similare 5 sesiuni de joc din BD
- Output: lista de elemente similare

=> Recomanda quest-urile
jucate de cei K vecini

Metrice de Distanța

Cum se măsoară similitudinea în spațiul caracteristicilor

Metrice de distanta folosite in KNN

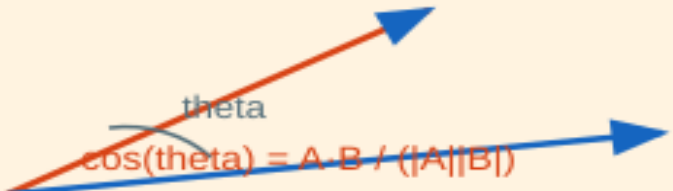
Distanța Euclidiană



A diagram showing two points, one blue and one red, connected by a dashed line labeled d .

$$d = \sqrt{\sum (x_i - y_i)^2}$$

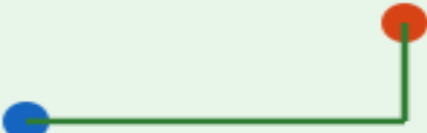
Similaritate Cosinus



A diagram showing two vectors, one red and one blue, originating from the same point. The angle between them is labeled θ . The formula for cosine similarity is shown below the vectors.

$$\cos(\theta) = \frac{A \cdot B}{(|A||B|)}$$

Distanța Manhattan



A diagram showing two points, one blue and one red, connected by a green line that moves horizontally and then vertically, representing the Manhattan distance.

$$d = \sum |x_i - y_i|$$

Distanța Hamming

$A = [1, 0, 1, 1, 0]$

$B = [1, 1, 1, 0, 0]$

$[=, X, =, X, =]$

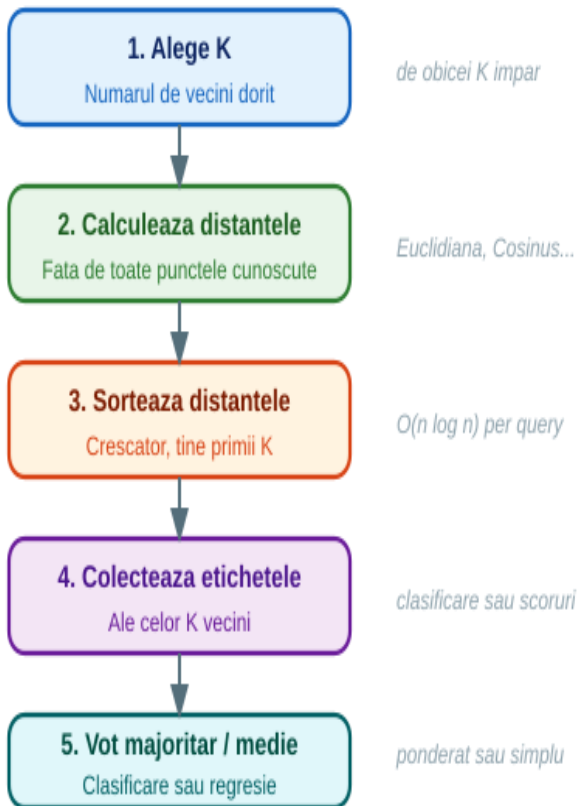
$d_H = 2$ (poziții diferite)

Alegerea metricii potrivite in jocuri

Metrica	Cand se foloseste	Exemplu in jocuri
Euclidiană	Date numerice continue (viteza, timp, scor)	<i>Profiluri jucator cu toate attributele continue</i>
Cosinus	Frecventa actiunilor, stiluri de joc (vectori pozitivi)	<i>De cate ori jucatorul foloseste fiecare abilitate</i>
Manhattan	Date cu unitati diferite, robuste la valori extreme	<i>Mix: timp(s) + scor(0-100) + nivel(1-50)</i>
Hamming	Preferinte binare (on/off), flags de comportament	<i>Sunet on/off, tutoriale completate, mod dificultate</i>
Minkowski	Generalizare: $p=1$ Manhattan, $p=2$ Euclidiană, $p=\infty$ Chebyshev	<i>Hiperparametru p ajustat prin validare incrucisata</i>

Algorithmul KNN

Algoritmul KNN

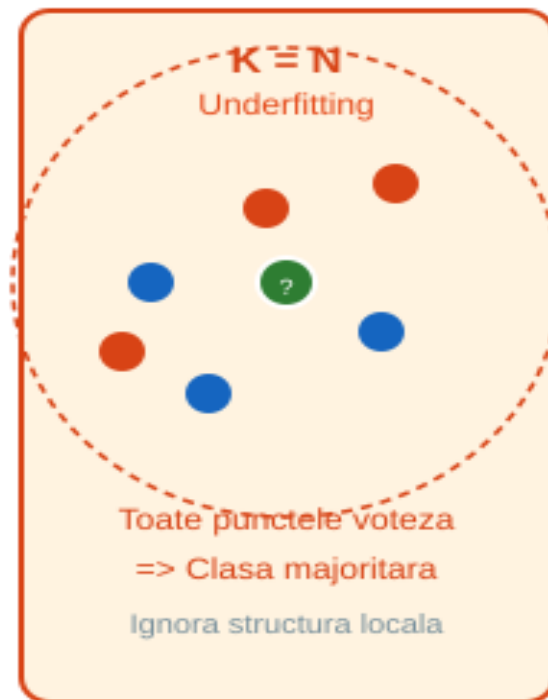
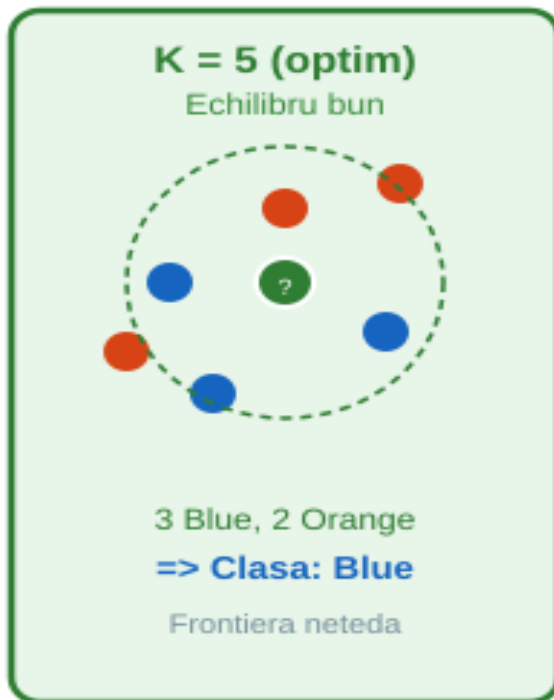
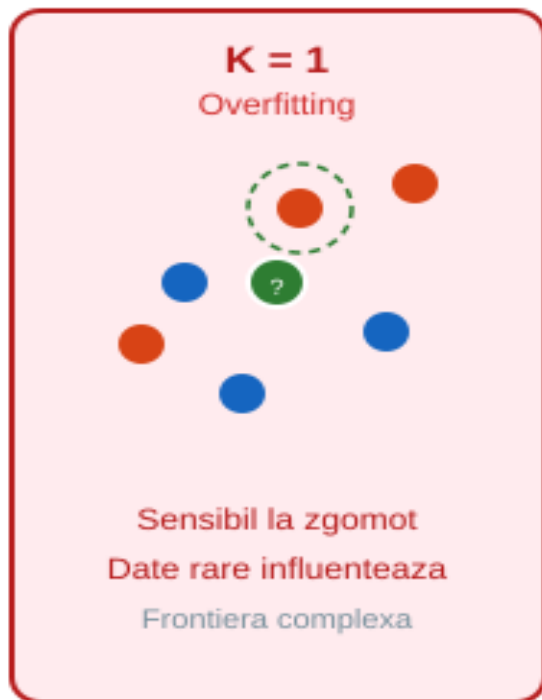


```
function KNN(Q, D, K):  
    distances = []  
    for x in D:  
        distances.append(dist(Q, x))  
    sort(distances)  
    return top_K(distances)
```

Complexitate timp

- Antrenare: $O(1)$ - lazy learner!
- Predictie: $O(n * d)$ brut
- n = nr. de exemple
- d = nr. de dimensiuni
- Stocare: $O(n * d)$ - intregul dataset

Alegerea valorii optime K

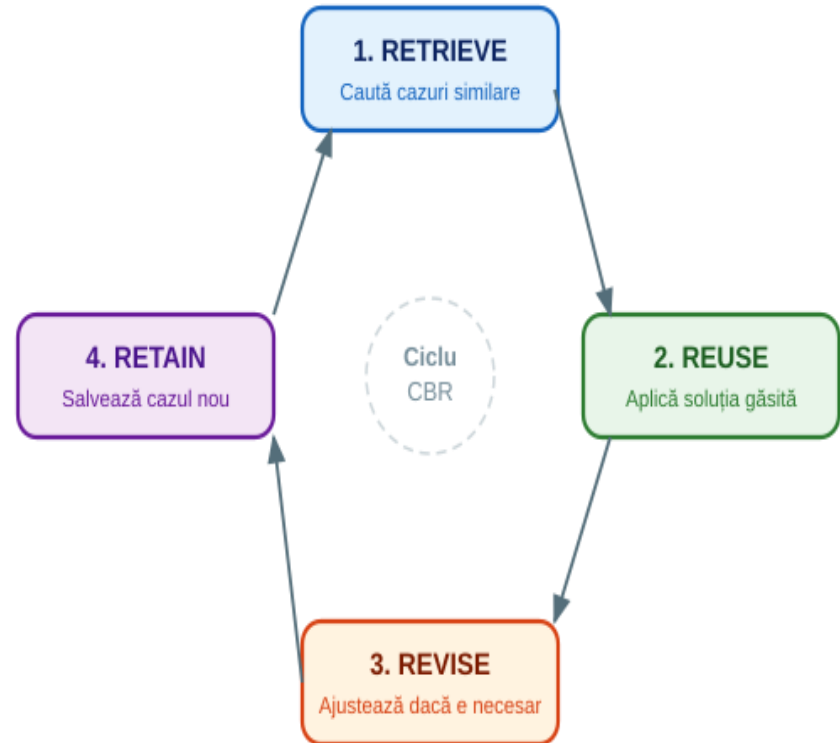


Case-Based Reasoning

CBR sistemul rezolva probleme noi reutilizând solutii din situatii similare anterioare. AI-ul **salveaza, compara si adapteaza** experiente anterioare.

Aplicare in jocuri

- AI care se adapteaza la tiparul de atac al jucatorului
- Inamici care "retin" cum a câstigat jucatorul ultima data
- NPC-uri cu comportament emergent bazat pe interactiuni anterioare
- Sisteme de naratiune adaptiva cu memorie istorica



CBR - Structura unui caz

Anatomia unui caz

- Context (problema): situatia din joc | Solutia aplicata: actiunea AI | Rezultatul: a functionat sau nu?

1 RETRIEVE

Cauta in baza de cazuri cel mai similar cu situatia curenta. Foloseste algoritmi de similitudine (Euclidean Distance, KNN).

2 REUSE

Aplica solutia gasita (sau o varianta a ei) situatiei curente. Daca e identica, solutia se aplica direct.

3 REVISE

Verifica daca solutia a functionat. Daca nu, ajusteaza (adaptare locala sau globala a solutiei).

4 RETAIN

Salveaza cazul nou (context + solutie + rezultat) in baza de cazuri pentru utilizari viitoare.

CBR - Algoritmi de similitudine

Distanța Euclidiană între vectori de caracteristici

$$d(\text{caz_nou}, \text{caz_vechi}) = \sqrt{\sum w_i \cdot (x_i - y_i)^2}$$

x_i = caracteristica i din cazul nou | y_i = caracteristica i din cazul vechi | w_i = pondere

K-Nearest Neighbors (KNN) pentru regasire

Selectează cele mai apropiate K cazuri din baza de date, ponderează soluțiile în funcție de distanță → soluție compozită.

CBR - Avantaje, Limitari si Reprezentare

Avantaje

- Se adapteaza fara reguli scrise manual
- Reutilizeaza eficient cunostintele
- Evolueaza cu istoricul jucatorului
- Bun pentru jucatori umani variabili

Limitari

- Necesita o baza de cazuri diversificata
- Mecanismele de adaptare pot fi complexe
- Risc de overfitting pe cazuri nerelevante
- Scalabilitate la baze mari de cazuri

Reprezentarea unui caz in jocuri

- { context: {scor: 850, vietii: 1, nivel: 3, timp_reactie: 0.8s}, actiune: "activeaza_elite_AI", rezultat: {jucator_avansat: true, rating: +1} }

Clusterizare

K-Means

K-Means grupeaza jucatorii sau situatiile de joc in K clustere bazate pe similaritate, pentru a permite AI-ului sa aplice strategii diferite fiecarui grup.

Segmentare jucatori

Casual / Mediu / Expert

Clustere de comportament

Agresiv / Explorator / Stealth

Clustere de dificultate

Zone din joc grupate dupa complexitate → generare procedurala

K-Means

1 Initializare

Alege K centroizi aleatori din setul de date (sau K-Means++)

2 Atribuire

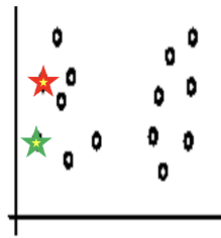
Fiecare punct este atribuit celui mai apropiat centroid (distanța Euclidiană)

3 Actualizare

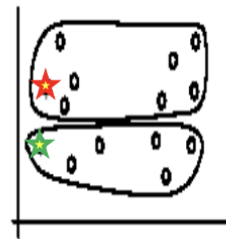
Recalculează centroizii ca media tuturor punctelor din cluster

4 Convergență

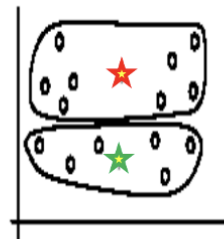
Repetă pașii 2-3 până când centroizii nu se mai modifică semnificativ



Alegere aleatoare a centroizilor



Iteratia 1



Iteratia 2

n = număr elemente | k = număr clustere | t = număr iterații → algoritm liniar (k, t mici)

K-Means - Avantaje

✓ Avantaje K-Means

- **Usor de implementat si de inteles**
- Eficienta computationala: $O(t \cdot k \cdot n)$ – liniar in n , ideal pentru seturi mari de date
- Scalabil la milioane de puncte cu variante optimizate
- Rezultate bune pentru clustere bine separate si sferice
- Usor de paralelizat pe GPU pentru aplicatii real-time in jocuri

K-Means - Dezavantaje



Dezavantaje K-Means

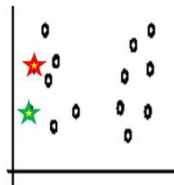
- **K trebuie specificat de la inceput**
- Nu stim numarul ideal de clustere – necesita tehnici suplimentare (Elbow Method, Silhouette Score)
- **Sensibil la outlieri (valori extreme)**
- Un jucator cu comportament atipic poate distorsiona centroidul intregului cluster
- **Sensibil la initializarea centroizilor**
- Alegerea slaba a centroizilor initiali → convergenta la un minim local suboptim (solutie: K-Means++)

K-Means

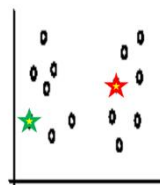
Aceleasi date, initializari diferite → rezultate diferite

✓ Initializare buna (K-Means++)

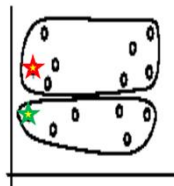
- Centrozii sunt alesi departati unii de altii
- Convergenta rapida la solutie optima
- Clustere clare si reprezentative
- K-Means++ garanteaza $O(\log k)$ -aproximare



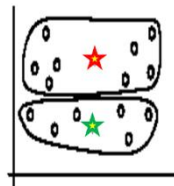
Alegere aleatoare a centrozilor



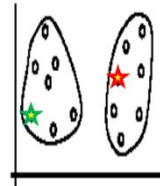
Alegere aleatoare a centrozilor



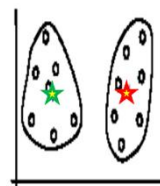
Iteratia 1



Iteratia 2



Iteratia 1



Iteratia 2

Fuzzy K-Means

Fuzzy K-Means extensie a K-Means in care fiecare punct de date poate apartine mai multor clusterelor simultan, cu un **grad de apartenenta** (0–1) in loc de o apartenenta binara.

K-Means clasic

- Fiecare jucator → exact un cluster
- Ex: Jucatorul X este 100% "Expert"

Fuzzy K-Means

- Fiecare jucator → apartine tuturor clusterelor cu grade diferite
- Ex: 70% Expert, 25% Mediu, 5% Casual

Fuzzy K-Means - suprapunerea clusterelor

Principiu fundamental

- K-Means: fiecare exemplu apartine unui singur cluster (hard clustering)
- Fuzzy K-Means: fiecare exemplu apartine tuturor clusterelor, cu grade $\mu_{ij} \in [0,1]$, unde $\sum_j \mu_{ij} = 1$

Gradul de apartenenta al exemplului i la clusterul j :

$$\mu_{ij} = \left(\sum_{k=1}^c \left(\frac{\|x_i - v_j\|}{\|x_i - v_k\|} \right)^{\frac{2}{m-1}} \right)^{-1}$$

- μ_{ij} gradul de apartenenta al exemplului i la clusterul j
- x_i exemplul i
- v_j centroidul clusterului j
- c numarul de clustere
- m fuzziness ($m > 1$)

Fuzzy K-Means

1 Initializare

Stabileste K si $m > 1$ (fuzziness). Initializeaza centroizii aleator.

2 Calculul μ_{ij}

Calculeaza gradul de apartenenta al fiecarui exemplu la fiecare cluster.

3 Actualizare centroizi

Recalculeaza centroizii ca medie ponderata de gradele de apartenenta.

4 Convergenta

Repeta pasii 2-3 pâna când μ si centroizii nu se mai modifica semnificativ.

Fuzzy K-Means - Fuzziness

$m \rightarrow 1$ (aproape K-Means)

- Apartenența aproape binară (0 sau 1)
- Clustere bine delimitate
- Comportament similar cu K-Means clasic
- Adecvat: jucători cu profil clar definit

$m = 2$ (valoare tipică)

- Echilibru explorare / precizie
- Grad de apartenență moderat
- Cel mai des utilizat în practică
- Adecvat: profiluri mixte de jucători

$m \rightarrow \infty$ (fuzzy maxim)

- Toate gradele $\mu_{ij} \rightarrow 1/K$
- Clustere complet suprapuse
- Nicio informație de separare
- Evitați această extremă

Naratiune Personalizata si RLHF

Povesti adaptate jucatorului si feedback uman in antrenamentul AI

Naratiune personalizata

Naratiunea personalizata presupune adaptarea povestii unui joc in functie de alegerile, stilul si actiunile jucatorului, folosind modele dinamice de decizie si invatare automata.

Libertate totala

Jucatorul exploreaza orice directie, povestea urmareste

Povesti emergente

Naratiuni unice, nerepetabile, generate contextual

NPC-uri vii

Personaje cu memorie si reactii autentice la comportamentul jucatorului

Adaptare in timp real

Tonul, ritmul si continutul povestii se ajusteaza continuu

Naratiune AI

✓ Avantaje ale naratiunii AI

- Libertate totala de explorare si decizie
- Povesti neasteptate si emergente, unice
- Joc neliniar cu experiente reel personalizate
- Adaptare la stilul (agresiv, diplomatic, stealth)
- Creste imersiunea si angajamentul pe termen lung

⚠ Limitari si provocari

- Generarea poate duce la inconsistente logice
- Modelele nu inteleg cauzalitatea profunda
- Control continut (evitare continut inadecvat)
- Costuri ridicate pentru rulara LLM-urilor
- Dificil de testat si de garantat calitatea

Tehnici de imbunatatire a naratiunii AI

Memorarea alegerilor

AI-ul retine alegerile anterioare ale jucatorului si le referentiaza in dialoguri viitoare, creând continuitate narativa.

Filtre de continut

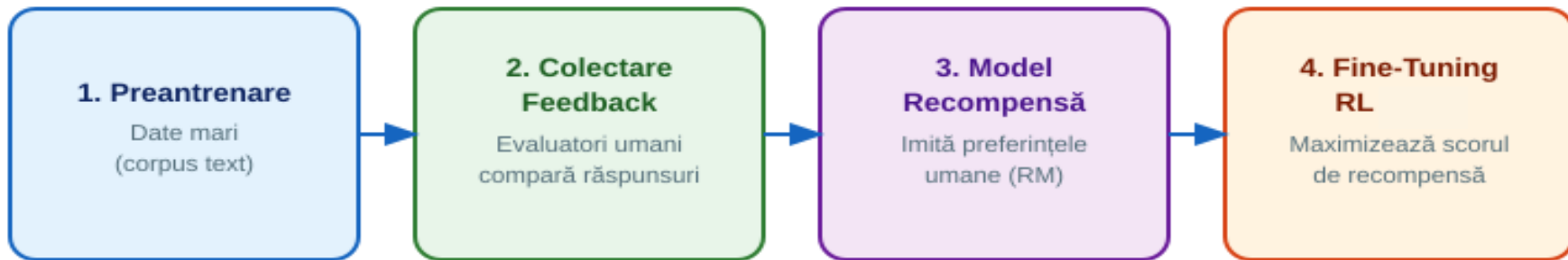
Layer de moderare care detecteaza si blocheaza generarea de continut neadecvat sau inconsistent cu universul jocului.

RLHF

Reinforcement Learning from Human Feedback – modelul este antrenat cu feedback uman pentru a genera naratiuni mai bune si mai aliniate.

Reinforcement Learning from Human Feedback

RLHF tehnica de antrenare a modelelor AI in care **feedback-ul uman** este folosit pentru a ghida procesul de invatare prin recompensa, cu scopul de a obtine un comportament mai sigur, mai util si mai aliniat cu valorile umane.



Reinforcement Learning from Human Feedback

Problema modelelor pre-antrenate

- Modelele mari de limbaj (LLM) pot genera raspunsuri coerente dar nu neaparat corecte, utile sau sigure
→ RLHF corecteaza comportamentul prin evaluari umane

Beneficii RLHF

- Evita generarea de continut periculos, neadecvat sau eronat
- Aliniaza modelul la preferintele reale ale utilizatorilor
- Imbunatateste utilitatea practica a raspunsurilor
- Permite controlul tonal si stilistic al dialogurilor NPC

RLHF

Avantaje

- Calitate superioara a raspunsurilor (util, sigur, relevant)
- Adaptare la nevoile si valorile umane
- Control fin al comportamentului AI
- Potential in alinierea AI la etica si siguranta
- Scalabil – se poate aplica oricarui LLM

Limitari si provocari

- Feedback-ul uman poate fi subiectiv si inconsistent
- Necesita resurse mari pentru colectarea evaluarilor
- Posibil overfitting la preferintele evaluatorilor